EC 69212: Design Lab (Networks)

Topic: Exploring Principles of Congestion in TCP

Week: 10 *Instructor: Amitalok J. Budkuley/Aneek Adhya* *Semester: Autumn 2026*

Lab Plan Overview

- Part I: Baseline Network Exploration
- Part II: Congestion Control Design
- Part III: MultiFlow congestion control
- Part IV: (Challenge) From Congestion control to *Congestion Avoidance*

Prelude

Modern networks must handle multiple senders sharing limited resources. One of the central challenges is *congestion control,* which determines how fast a sender can transmit data without overwhelming the network. In this lab, we simulate a simplified network consisting of a sender, a bottleneck queue, and a receiver. The sender transmits packets based on a congestion window (*cwnd*), which limits the number of packets it can inject into the network during each round-trip time (RTT).

The bottleneck queue accumulates incoming packets and serves them at a fixed rate μ per time unit. The queue has a finite buffer size K, and any packets arriving when the buffer is full are dropped. The evolution of the queue length q can be described by:

$$q_{t+1} = \max\left(q_t + \text{arrivals}_t - \mu, 0\right)$$

where $arrivals_t$ is the number of packets sent in that time step. *Packet drops act as an important signal of congestion in the network.*

A naive sender might simply increase *cwnd* linearly, for example:

$$cwnd \leftarrow cwnd + 1$$

However, this approach often leads to unstable behavior: queues may grow excessively, causing frequent packet drops, and multiple flows may compete unfairly for bandwidth. To address this, TCP employs *feedback-based congestion control strategies.* One common principle is *Linear Increase, Faster-Than-Linear (FTL) Decrease* or LI-FTLD: the sender increases its window slowly when there is no packet loss, and decreases it sharply upon detecting congestion:

Think: 'slow climb when things are okay, quick drop when congestion hits.'

This produces the characteristic *sawtooth pattern* in *cwnd* and stabilizes queue occupancy below the buffer limit.

Multiple Flows: When multiple flows share the same queue, each reacts independently to congestion signals. The principle of *linear increase and faster-than-linear decrease (LI-FTLD)* helps promote fair sharing of bandwidth. Naive strategies, by contrast, may allow one flow to dominate. Through this lab, you will explore the behavior of a naive sender, implement your own congestion control strategies, and finally study LI-FTLD and fairness among multiple competing flows. By experimenting with queues, window updates, and packet loss, you will develop an intuition for how TCP maintains stability, efficiency, and fairness in real networks.

Before starting, consider these questions:

1. What is congestion and how should a sender detect congestion and decide to reduce (or increase) its sending rate?
2. How should the sender safely increase its rate to efficiently use available bandwidth?
3. How do multiple senders sharing a queue affect fairness and stability?
4. How might variability or randomness in packet arrivals impact network behavior?

In a major variation from earlier sessions, in this experiment, we will begin with a simple example code. You will be required to work on this basic skeleton and propose your solutions, and hopefully, discover some fun principles of congestion control.

Optional challenge: If you finish early, you can explore congestion avoidance strategies like RED, which aim to provide early feedback before packet drops occur, smoothing queues and improving fairness.

Buckle up!

==========

Part I: Baseline Network Exploration

1. Aim

Understand congestion in networks by simulating a sender interacting with a bottleneck queue. Observe naive rate increase and its effect on queue and drops.

2. System Model

Sender → Bottleneck Queue → Receiver

Time Abstraction (RTT) We model time in discrete steps, where each step corresponds to one Round-Trip Time (RTT). An RTT represents the time taken for a packet to travel from sender to receiver and for an acknowledgment (ACK) to return.

In this simplified model, feedback about congestion (e.g., packet drops) is assumed to be available at the end of each RTT. Thus, the sender updates its congestion window (`cwnd`) once per time step based on observed network conditions.

At each time step (1 RTT):

- Sender transmits *cwnd* packets.

  Sending Model Simplification: In real TCP, the congestion window (`cwnd`) limits the number of packets that can be "in flight" (i.e., sent but not yet acknowledged). For simplicity, we approximate this by assuming that in each RTT, the sender injects `cwnd` packets into the network.

  Thus, `arrivals = cwnd` represents the number of packets entering the bottleneck queue during that time step.

- Queue accumulates packets
- Link serves μ packets
- Buffer size is limited to K
- Excess packets are dropped

3. Key Variables

- *cwnd*: congestion window
- q: queue size
- μ: service rate
- K: buffer size
- drops: packets dropped

4. Task A: Run the Baseline System

Use the provided MATLAB code:

```
% TCP CONGESTION CONTROL LAB - STARTER CODE

clear; clc; close all;

T = 300;
mu = 50;
K  = 100;

cwnd = 1;
q = 0;

cwnd_trace = zeros(1,T);
q_trace    = zeros(1,T);
drop_trace = zeros(1,T);

for t = 1:T

    arrivals = max(cwnd,0);

    q = q + arrivals;

    served = min(q, mu);
    q = q - served;

    if q > K
        drops = q - K;
        q = K;
    else
        drops = 0;
    end

    loss_event = (drops > 0);

    % BASELINE (no congestion control)
    cwnd = cwnd + 1;

    cwnd = max(cwnd,1);

    cwnd_trace(t) = cwnd;
    q_trace(t)    = q;
    drop_trace(t) = drops;

end

figure;
subplot(3,1,1); plot(cwnd_trace); title('cwnd');
subplot(3,1,2); plot(q_trace); title('Queue');
subplot(3,1,3); plot(drop_trace); title('Drops');
```

Notes:

- **Loss Signal Assumption:** We model congestion detection using an idealized loss signal: if any

packets are dropped during a time step, the sender immediately detects this as a congestion event.

In real TCP, packet loss is inferred indirectly through mechanisms such as duplicate ACKs or timeouts, and may be delayed or noisy. Here, we assume perfect and instantaneous feedback to focus on congestion control behavior.

5. Task B: Explore the Baseline Behavior

- Vary μ and K (e.g., $\mu = 30, 50, 70$, $K = 50, 100, 200$). Observe changes in queue and drops.
- Introduce slight randomness in arrivals: arrivals = max(cwnd + randi([−1,1]),0);
- Record:
  - Maximum and average queue
  - Maximum and average drops
  - Maximum and average *cwnd*

6. Observations and Reflection

Answer:

1. Does the queue stabilize or keep growing? What does *stability* mean to you?
2. How does *cwnd* growth relate to queue behavior?
3. Is increasing the sending rate always beneficial?
4. How do random arrivals affect stability? Comment clearly.

==========

Part II: Congestion Control Design

1. Objective

Modify the system to make it stable and efficient. Explore different congestion window (cwnd) backoff rules.

2. Task C: Think Before Coding

Before coding:

- When should the sender reduce its rate? How much any packet drop, if any, is sufficient?
- How should the sender increase its rate?
- Should increase and decrease be symmetric?
- Propose at least two potential rules for cwnd adjustment.

3. Task D: Implement and Test Own Rules

- Replace the baseline rule: cwnd = cwnd + 1;

Motivation: Observe what happens when the sender is "naive" and ignores congestion. How unstable can the queue become?

- Test at least two different backoff/increase rules, such as:

- Linear decrease on loss, linear increase *Motivation: See if proportional adjustments prevent the queue from growing too fast or dropping too many packets.*
- Multiplicative decrease, linear increase *Motivation: Explore sharper reactions to congestion. Does cutting back more aggressively help stabilize the network?*
- Threshold-based reduction *Motivation: Investigate if reacting to high queue occupancy rather than every drop can reduce oscillations and improve fairness.*

Note: Your goal is not just stability, but also efficiency (high utilization) and fairness.